

Analysis of Predictive Factors for Book Rating Levels

1st Christian Putra Dwiyan Setyo Pradana
Computer Science Department
Bina Nusantara University
Jakarta, Indonesia
christian.pradana@binus.ac.id

2nd Dideo Setia Simangunsong
Computer Science Department
Bina Nusantara University
Jakarta, Indonesia
dideosimangunsong@binus.ac.id

3rd Herrick Fabian
Computer Science Department
Bina Nusantara University
Jakarta, Indonesia
herick.fabian@binus.ac.id

4th Nadzla Andrita Intan Ghayatrie
Computer Science Department
Bina Nusantara University
Jakarta, Indonesia
nadzla.ghayatrie@binus.ac.id

Abstract—This study designs and implements a data engineering workflow using a public book dataset obtained from an open source platform (Kaggle). The main objective of this workflow is to prepare the data for analysis to build a machine learning model that can predict book rankings (high/low). The raw data is processed using the Extract, Transform, Load (ETL) method through Pentaho Data Integration[1]. A star schema model consisting of fact tables and dimension tables is designed to support reporting and analysis.

Keywords—Data Engineering, ETL, Star Schema, Data Transformation, Data Analytics

I. INTRODUCTION

A. Data Source and Data Generation

Data used for this research is a public dataset “Goodreads-books” from open source dataset platform “kaggle”. This data contains more than 11,000 rows about books and consist of some type of variable such as numerical variables(num_page, rating_counts, text_reviews_count, average rating) and categorical variables(bookID, title, authors, isbn, isbn13, language_code, publisher) and date data(publication_date).

B. Business and Analytical Relevance

The shift from a typical Extract, Transform Load (ETL) framework to a predictive classification model using a Star Schema provides deep strategic benefits to today’s publishing industry. In a highly competitive and saturated market, the ability to predict whether a book will receive a “High” or “Low” rating based on its structural metadata is far more than an academic exercise; this model can help alleviate financial risk. By predicting book popularity before publication, publishers can optimize the allocation of their resources quantitatively, adjust first print runs dynamically, and build highly selective campaigns.

Additionally, this forecasting ability is important for spotting titles that are likely to receive early favourable reviews that drive lucrative pre-order momentum. Outside of conventional

publishing, digital reading platforms also need correct rating predictions to discard bad suggestions. This allows them to offer a highly relevant piece of content (i.e. a book), which keeps the reader engaged. In turn, this engagement has a direct impact on consumer purchasing behaviour.

More analytically, this is a step forward from conventional reporting of data. Business intelligence (BI) dashboards are generally considered to be limited to retrospective, descriptive analytics, thereby providing only a view into the past. By implementing the Random Forest classification algorithm, the analytical pipeline emerges as a dynamic forward-looking predictive engine that can map complex non-linear relationships and discover hidden patterns without the need for pre-programmed rules. The algorithm, in ranking the predictive ability of features, reveals powerful reader insights. Significantly, the model’s predictive weight is explained mostly by structural variables, particularly page count and publication year, while high-cardinality categorical features, such as author and publisher names, clearly have marginal predictive power. This mathematically contradicts traditional understandings in the industry by demonstrating that a reader’s cognitive commitment to the book’s length will often be a more valid predictor of rating success than nominal identity.

II. DATA SOURCE AND DATA GENERATION

The dataset used in this project is the “Goodreads-books” dataset, which sourced from Kaggle (<https://www.kaggle.com/datasets/jealousleopard/goodreadsbooks/data>). The data is extracted and processed from a CSV file format.

- a. The nature of the data and its context
It contains over 11,000 records of books from the Goodreads platform, consisting of numerical variables such as page count and rating counts, categorical variables such as book title, author, publisher, and ISBN, and date data like the publication date.
- b. The relevance of the data to the intended analytical or business use case

This dataset is highly relevant for building a predictive classification model to forecast whether a book will receive a "High" or "Low" rating. From a business perspective, these predictions allow publishers to mitigate financial risk. From the analytical view, they can improve reading recommendations by uncovering hidden patterns such as that a book's page count often influences reader rankings more than the author's or publisher's reputation.

III. DATA INGESTION AND DATA STORAGE

A. Data Ingestion

Data Ingestion Process is done on Pentaho Data Integration. The data is from a static csv file so we implement full load strategy that pull all of the data from the source to Pentaho Data Integration to start the transformation process[2].

B. Data Storage

To ensure the data integrity, we use 2 type of storage to store the data:

1. File Storage: Store the raw csv file in the local directory to keep the source data and to act as a backup if the ETL process is failed so the raw data is not modified.
2. Transformed Storage: After the ETL process is done, the data that has already been split up by their dimensions and turned into star schema stored in an OLAP database to ease and fasten the query.

IV. DATA TRANSFORMATION AND MODELING

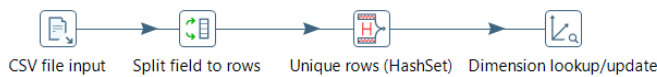
A. Data Transformation

To ensure the data is usable for the predictive model, data cleaning and checking is a must step.

1. Data Cleaning: Removing duplicate and handling inconsistent by standardized the column/variable that have more than one format.
2. Data Validation: Ensure the data type of each variable is correct/relevant .

To transform the author dimension which contains the authors name, the authors name needs to be separated for the book with more than 1 author. For example, a book contains 2 authors J. K. Rowling/Mary GrandPré is separated into 2 rows each containing only 1 author and adds surrogate keys before inserting the data into the database.

Fig 1: author dimension transformation



For the book dimension, we only need to validate and ensure the book's page data type is correct and add surrogate keys before inserting the data into the database.

Fig 2: book dimension transformation



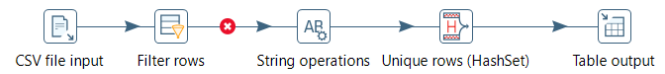
Publisher dimension only needed to add a surrogate key then add the data into the database.

Fig 3: publisher dimension transformation



Language dimension only needed to standardize the format by uppercase all the language code and add surrogate key before adding the data into the database

Fig 4: language dimension transformation



For time dimension, we separate the date month, date, and year to make 3 new columns before adding the surrogate key and adding the data into the database.

Fig 5: time dimension transformation



For the fact table, we carry out the transformation that has been done in the previous tables so that the data is the same and when the data is looked up, it is valid, then add a database lookup to add surrogate keys from each dimension that has been created as a foreign key so that it can refer to the relevant table.

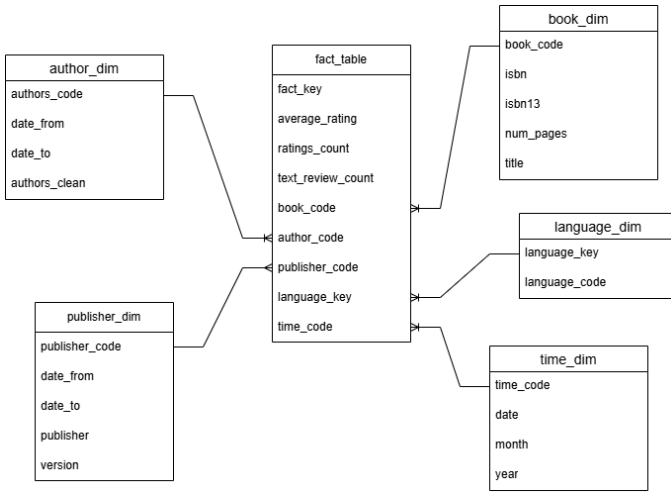
Fig 6: fact table creation



B. Data Modeling

The process of making the star schema is done at the same time as data transformation. Star schema is used to optimize the performance by reducing the need to join tables and avoiding duplicates attributes, and this structure makes it

easier to extract clean and structured features to train the Machine Learning model (ML) [3]. The dataset is divided into 5 dimensions and each dimension has a surrogate key to refer to the fact table and store information data from related dimensions such as author dimension that saves author name, book dimension that saves title, isbn, isbn 13, and the page number, publisher dimension that save publisher name, language dimension that save language code, and time dimension that save date, month, and year of the published date, and the fact table that save aggregation and surrogate key for each dimensions. Scd type 0 is used when the dimension stores static data such as time dimensions and language dimensions. Scd type 1 is used when the dimension stores static data such as time dimensions and language dimensions. Scd type 1 is used when the dimension stores the most recent data without storing data history for example book dimension, and Scd type 2 to store latest data and also the data histories such as author dimension and publisher dimension.



V. AUTOMATION AND ORCHESTRATION

To automate the ETL pipeline we implemented a pentaho job to run each transformation automatically instead of running each transformation one by one. The orchestration runs sequentially starting from author dimension, book dimension, publisher dimension, language dimension, time dimension, and lastly fact table.

Fig 7: Automation using pentaho job



VI. DATA CONSUMPTION

A. Modelling Approach

At the data consumption stage, data that has been cleaned and modeled into a Star Schema undergoes further processing using a Machine Learning (ML) approach, rather than simply being presented via conventional Business Intelligence (BI) dashboards like Power BI. While interactive dashboards are ideal for presenting descriptive analytics about historical metrics, they have limitations when it comes to uncovering predictive patterns. Therefore, this project implements the Random Forest classification algorithm to increase analytical capabilities. The objective of this modeling is to determine the probability and rank hidden predictive factors influencing average book ratings. This yields insights that are difficult to extract through manual interpretation of standard graphical visualizations alone.

The selection of an ML classification model instead of Power BI dashboard is based on ML's ability to exceed the limitations of static reporting and enable predictive decision-making. According to Bharadiya (2023) [4], the integration of ML capabilities significantly surpasses traditional BI tools. ML not only analyzes historical data but also automates the detection of future trends within complex datasets. This analysis works without the need for explicit rule programming (hard-coding) [1]. In the context of this Goodreads data processing, Random Forest modeling enables the data pipeline to evaluate non-linear relationships among various numerical and categorical variables to predictively determine the probability of the target rating class (Low or High).

B. Data Preparation

The primary target for classification is the average book rating ('average_rating'), which was converted into a binary variable. Books with a rating of ≤ 3.9 were categorized as the "Low" class (represented by the value 0), while books with a rating of ≥ 4.0 were categorized as the "High" class (represented by the value 1).

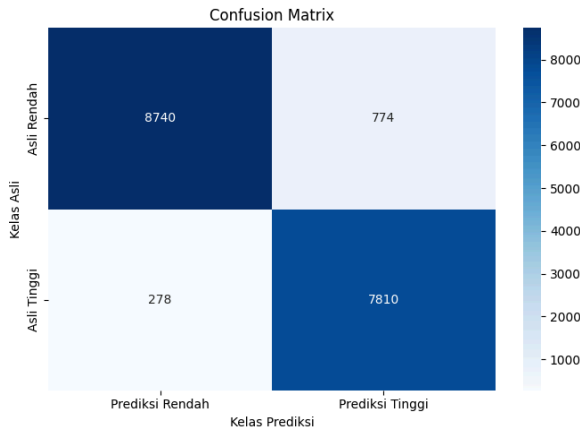
Regarding class distribution, the dataset exhibits a relatively balanced proportion: 54.05% for the "Low" class and 45.94% for the "High" class. The predictor features employed include numerical variables such as release year ('year') and page count ('num_page'). For the categorical variables including publisher ('publisher'), language code ('language_code'), and author ('author_clean'). The dataset was subsequently split into 80% training data and 20% testing data.

C. Model Evaluation

Fig 8: Classification Report

	precision	recall	f1-score	support
0: Rendah	0.97	0.92	0.94	9514
1: Tinggi	0.91	0.97	0.94	8088
accuracy			0.94	17602
macro avg	0.94	0.94	0.94	17602
weighted avg	0.94	0.94	0.94	17602

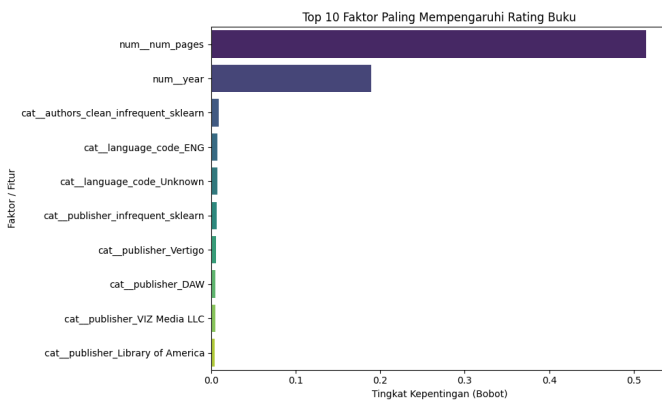
Fig 9: Confusion Matrix



The classification model's performance was evaluated using standard metrics such as accuracy, precision, recall, and f-1 score. The developed Random Forest model achieved an overall accuracy of 94% on the test data. According to the confusion matrix, the model demonstrated excellent sensitivity in detecting the high-rating class, yielding 7,810 correct predictions and only 278 incorrect ones (False Negatives). Conversely, the model was slightly more prone to false positives, with 774 low-rated books incorrectly predicted as high-rated, while 8,740 low-rated books were correctly classified.

D. Feature Analysis and Insight

Fig 10: Barplot of Feature Importance



Feature importance analysis extracted from the Random Forest algorithm reveals that quantitative variables has greater predictive influence than categorical variables. The factor playing the most significant role in determining book ratings is the book's thickness or page count ('num_num_pages'), which holds the highest importance weight at 0.514. The second most influential predictor is the year of publication ('num_year'), with a weight of 0.189.

On the contrary, features with categorical feature such as author identity ('cat_authors_clean'), language code ('cat_language_code'), and publisher name ('cat_publisher'), contribute very few to the model, with each recording an importance weight below 0.01. This indicates that when assigning ratings, audiences are more influenced by the substance of the content (represented by page count) and the era of publication than by the reputation of the publisher or author.

Fig 11: Barplot of Book Probability Getting High Rated Based on Number of Pages

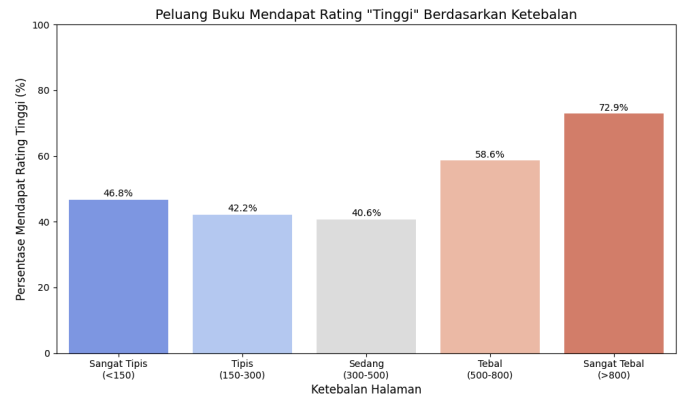
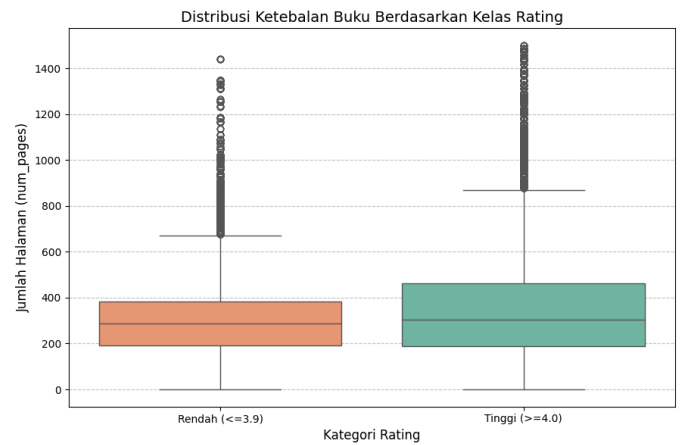


Fig 12: Boxplot of Books Pages Distribution Based on Rating Class

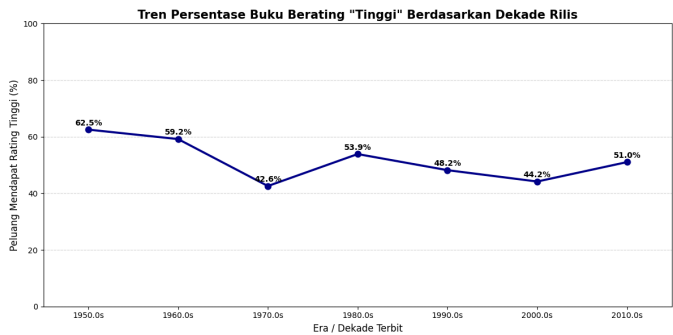


Further analysis shows a clear positive relationship between the number of pages and the chance of getting a high rating. Books in the "Very Thin" category (less than 150 pages) have

a 46.8% chance of receiving a high rating. This percentage is slightly lower for "Thin" books (150–300 pages) at 42.2% and "Medium" books (300–500 pages) at 40.6%. However, the probability increases to 58.6% for "Thick" books (500–800 pages) and reaches 72.9% for "Very Thick" books (more than 800 pages).

The box plot also supports the result, showing that books in the High rating group (≥ 4.0) have a higher median page count and a wider spread of page numbers than books in the Low rating group (≤ 3.9). One possible explanation is that readers who spend the time to finish longer books are more committed and are therefore more likely to give positive ratings. This may reflect a type of readership survivorship bias, where only readers who enjoy the book are likely to finish it and leave a review

Fig 12: Trend of Books Probability Getting High Rated Based on Decade of Release



Besides page count, analyzing publication decades provides useful insights into how reader preferences have changed over time. The results show that books published in the 1950s had the highest probability of receiving a High rating at 62.5%, followed by books from the 1960s at 59.2%. After that, the probability decreased and reached its lowest point in the 1970s at 42.6%. It increased again to 53.9% in the 1980s but fell to 48.2% in the 1990s and 44.2% in the early 2000s. In the 2010s, the probability gradually increased again to 51.0%. One possible reason is that readers on platforms such as Goodreads often have a stronger appreciation for classic books. In contrast, modern books published after the 2000s face greater competition because many more books are released, making it harder to receive consistently high ratings.

VII. EVALUATION AND REFLECTION

The Random Forest model implemented in this data engineering pipeline produced robust and useful results. However, on review, certain limitations could be found when predicting the subjective human reception of a piece only based on structural metadata. The model's greatest strength is its remarkable classification performance of 94% overall accuracy on the unseen test data. This clearly shows that it outperforms basic linear models on complex literary text. The model demonstrated exceptional sensitivity acting as a very

reliable safety net for identifying revenue-generating potential since it successfully captured 7,810 high-rated books while missing only 278 books. The Random Forest architecture, being a non-parametric ensemble method, is capable of taking in data with both numeric and categorical features without needing heavy prior normalisation. Furthermore, it provides clear and explicit feature importance weights and avoids the black box nature of neural networks.

Of course, these upsides come at the price of model weaknesses inherent to their architecture. My pipeline suffered from heavy-tail bias, classifying 774 poorly rated books as if they were highly rated. This ratio of false positives shows how the underlying algorithm was unable to distinguish poorly written books of similar structure. (It should come as no surprise that books follow this power law.) Most importantly however, the entire pipeline used nothing but objective metadata. This means no consideration was given to prose quality, emotion invoked, pacing, etc. Another missed component was author and publisher information. The pipeline did not learn anything useful from these high cardinality nominal variables. It also did not take into account user-opinion of the community or user-to-user network effects.

Well that about sums up everything that didn't happen. The solutions to each bullet point above are rather succinct. In order to not take each book's rating as a singleton, my content based filtering pipeline should have merged with user-user collaborative filtering through matrix factorization. Doing this will allow the algorithm to better understand latent user preferences as well as the topology of the book community. To begin leveraging high cardinality categories, my data pipeline will need to target encode these variables. This will allow the model to better use an author/publisher's historical track record by converting categorical ratings into continuous mathematical indicators of past performance. In order to correct for heavy-tails and include subjects that catch the reader's eye, I can look towards implementing sequential error correction algorithms such as Extreme Gradient Boosting. I can also use a Computer Vision pipeline to turn book covers into visual feature columns.

REFERENCES

- [1] A. Yang, J. Wiratama, and S. F. Wijaya, "Empowering Data Transformation: Transforming Raw Data into A Strategic Planning for E-Commerce Success," vol. 6, no. 1, 2024. [Online]. Available: <https://journal-isi.org/index.php/isi/article/view/665>
- [2] A. D. Barahama, R. Wardani, and J. F. Lempas, "Utilization Extract, Transform, Load For Developing Data Warehouse In Education Using Pentaho Data Integration," vol. 2111, no. 1, 2021. [Online]. Available: <https://iopscience.iop.org/article/10.1088/1742-6596/2111/1/012030/pdf>
- [3] I. Thoib, B. P. Candra, F. B. Firmansyah, D. S. Nugraha, and N. Sururi, "Perancangan Data Warehouse Sebagai Penunjang Strategi Penerimaan Mahasiswa Baru," vol. 4, no. 1, 2025. [Online]. Available: <https://ejurnal.lkpkaryaprima.id/index.php/juktisi/article/view/346/218>

- [4] J. Bharadiya, "The Role of Machine Learning In Transforming"Business Intelligence," vol: 4, no, 1, 2023.
DOI: <https://doi.org/10.33545/27076571.2023.v4.i1a.60>